

Abstract

Keywords: Static Application Security Testing, Large Language Models,
C programming language, False Positives

Resumen

Keywords: Analizador de código estático, Modelo extenso de lenguaje,
Lenguaje de programación C, Falsos positivos

Índice

1. Introducción	7
2. Estado del arte	9
2.1. El funcionamiento de la pila de llamadas	9
2.2. El funcionamiento del montículo (<i>heap</i>)	9
2.3. Organización de la memoria interna de un proceso	10
2.4. CWEs de la pila de llamadas	11
2.5. Causas del desbordamiento de pila	11
2.6. CWEs del montículo	11
3. Metodología de análisis	13
4. Entrenamiento del gran modelo de lenguaje	15
5. Resultados recopilados	17
6. Conclusiones	19

Índice de figuras

1. Estructura de memoria de un proceso C/C++. Extraída de [5] 10
2. A la izquierda se tiene el código fuente, a la derecha, el contenido de la pila en el nivel más profundo de recursión. En este caso, el calculo se expande según se profundiza. En situaciones de recursividad muy profunda, aumenta el riesgo de desbordamiento de pila por agotamiento de espacio. Ejemplo extraído de Wikipedia [7]. 12

1

Introducción

El lenguaje de programación C es uno de los lenguajes más populares en la industria de desarrollo de software. Este lenguaje adquiere una notable presencia en sistemas en los que se necesita del acceso a bajo nivel al hardware, la eficiencia temporal y el control explícito de los recursos. Originalmente, C se planteó como un lenguaje de programación de sistemas para la familia de sistemas operativos tipo UNIX, caracterizado técnicamente como un lenguaje liviano, de propósito general y portable [1]. Una gran cantidad de aplicaciones de alto rendimiento y sistemas críticos se implementan en C y continúan en soporte.

Sin embargo, las características que popularizaron a C también son el fondo detrás de una gestión insegura de la memoria intrínseca al lenguaje. Esta gestión insegura propicia la aparición de errores en el manejo de la memoria, que, además de ser difíciles de detectar, constituyen una de las superficies de ataque más persistentes en los lenguajes de programación *memory-unsafe*.

Los lenguajes de programación *memory-unsafe* no cuentan, por defecto, con mecanismos de prevención de errores comunes de programación relacionados con la libertad en el acceso a memoria. Bajo esta libertad, el programador gestiona manualmente la memoria, pero esta libertad conlleva la aparición de distintos tipos de vulnerabilidades, entre las que figuran los desbordamientos de pila, las filtraciones de memoria y la corrupción de memoria.

Concretamente, las vulnerabilidades de corrupción de memoria constituyen el tipo de vulnerabilidad más frecuente. La sobre-escritura de metadatos del montículo, los punteros colgantes, las lecturas de memoria no inicializada, las liberaciones inválidas de memoria o la doble liberación de memoria son debilidades reconocidas por investigadores de seguridad y cuentan con sus respectivas técnicas de protección para abordarlas [2].

Cada una de las vulnerabilidades anteriormente mencionadas se recogen en la Common Weakness Enumeration (CWE), por lo que hacemos énfasis en que las fallas que se encuentran

en programas escritos en C no son errores individuales aislados, sino que son consecuencia del diseño estructural del lenguaje.

A la vista de este escenario, la detección de debilidades supone una necesidad con objeto de asegurar la calidad y diseñar software seguro. La literatura sobre el análisis de programas señala una reducción en el coste de corrección de vulnerabilidades si se detectan tempranamente. De hecho, facilita la implementación de prácticas de seguridad «*shift-left*» e intervenir antes de que los defectos lleguen a la etapa de despliegue o de explotación [3].

Los analizadores de código estático son herramientas que permiten la detección de vulnerabilidades en el código fuente a partir de la detección de patrones. Son herramientas de referencia en la industria de desarrollo software, por ejemplo en las canalizaciones de integración continua, dado que permiten identificar estas vulnerabilidades en las fases tempranas del ciclo de vida del software. Por el contrario, hay estudios empíricos que muestran sus limitaciones en la precisión, cobertura, explicabilidad y unas altas tasas de positivos en sus reportajes [4]. Por lo tanto, detectar estas vulnerabilidades asegurando una alta fiabilidad y utilidad sigue siendo un problema abierto.

Estado del arte

En el contexto de este trabajo profundizaremos en el análisis de vulnerabilidades relacionadas con la pila de llamadas, también llamada pila o *call stack* y el montículo, también llamado *heap*, en el lenguaje C. Ambas son dos regiones de memoria relevantes en la estructura de los programas en tiempo de ejecución. Expondremos el funcionamiento de ambas, y posteriormente explicaremos sus debilidades.

2.1. El funcionamiento de la pila de llamadas

En la pila de llamadas (*stack call*) se almacenan los datos relativos a las subrutinas de un programa. Cuando se invoca una subrutina, se crea en la pila un nuevo marco *stack frame*, que contiene sus parámetros, la dirección de retorno y sus variables locales. Al finalizar la subrutina, su marco se libera.

La pila sigue una estrategia LIFO (*last in, first out*): el último marco en crearse es el primero en destruirse. Por ello, las subrutinas más anidadas (las llamadas más recientes) son las últimas en alojarse pero las primeras en liberarse, de forma análoga a una pila de libros donde solo se puede retirar el libro de la cima.

La gestión de la pila se realiza mediante un puntero de pila. Al insertar un nuevo marco, el puntero se desplaza hasta el tope de dicho marco. Y al salir, el puntero retrocede hasta el tope del marco anterior (o al límite inicial si la pila queda vacía). Estos movimientos son automáticos y el programador no tiene control directo sobre ellos.

2.2. El funcionamiento del montículo (*heap*)

El montículo (*heap*) es una región de memoria dinámica; es decir, los datos almacenados en él se asignan durante el tiempo de ejecución del programa. A diferencia de la pila, los datos del

montículo deben gestionarse manualmente.

La gestión manual de la memoria comprende dos operaciones fundamentales: la petición y la liberación de bloques de memoria. Para solicitar un bloque de un número específico de bytes, se emplean las funciones `malloc` o `calloc`, que devuelven un puntero al inicio del bloque asignado. La liberación se realiza mediante la función `free`, a la que se proporciona como argumento el puntero del bloque que se desea liberar.

Al delegar la gestión de la memoria en el programador, los errores aparecen con una frecuencia mucho mayor. Uno de los más característicos son las **fugas de memoria** (memory leaks), que ocurren cuando no se libera un bloque de memoria tras cumplir su utilidad. En casos severos, puede llevar al agotamiento de los recursos (especialmente en dispositivos con capacidades limitadas), lo que puede derivar en una denegación de servicio [9].

2.3. Organización de la memoria interna de un proceso

Dentro de un proceso, la pila y el montículo comparten la misma región de memoria, con la particularidad de que ambas crecen en direcciones opuestas. Por un lado, la pila tiene un límite de tamaño impuesto por el sistema: en Linux puede crecer hasta 8 MB, mientras que en Windows el límite típico es de 1 MB. En cambio, el montículo puede crecer hasta agotar la memoria virtual disponible. En la figura 1 se observa esta región compartida dentro de la estructura de memoria de un proceso.

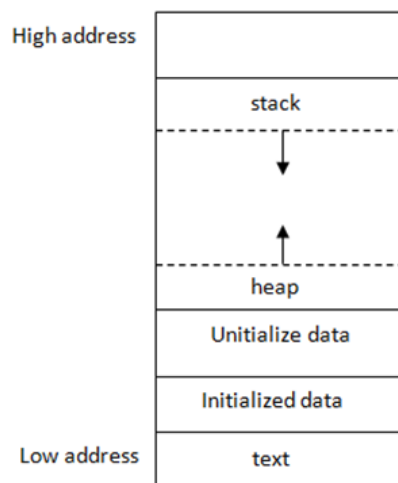


Figura 1: Estructura de memoria de un proceso C/C++. Extraída de [5]

2.4. CWEs de la pila de llamadas

Las debilidades recogidas en el catalogo CWE son fallos estandarizados conocidos por causar vulnerabilidades. En este apartado detallamos 5 fallos asociados a la pila de llamadas.

2.5. Causas del desbordamiento de pila

Generalmente, los desbordamientos de pila pueden atribuirse a dos causas, aquellos que tienen que se deben a función recursiva, que debido a la cantidad de datos que aloja con sus llamadas agota el espacio de la pila; y aquellos se dan al intentar almacenar variables excesivamente grandes en la pila.

Las funciones recursivas son aquellas funciones que se llaman a si mismas dentro de su propio código, estas funciones para evitar llamadas infinitas cuentan con un caso base, que es una condición que al cumplirse detiene las llamadas recursivas. Aunque hacemos énfasis

Un desbordamiento de pila puede deberse en una función recursiva. El primero es que haya una profundidad excesiva de llamadas, aunque todos los marcos contengan la misma cantidad de datos; y segundo, que haya un crecimiento en el tamaño de los marcos con la profundidad (por ejemplo, cuando cada marco nuevo contiene estructuras de datos de mayor tamaño que las anteriores), acelerándose así el ritmo de consumo de la pila. En ambos casos, tanto en funciones recursivas con casos base lejanos, como con marcos de tamaño creciente, se puede producir un desbordamiento. En la figura 2 puede verse un ejemplo de funciones recursivas en las que el tamaño del marco crece conforme aumenta la profundidad.

El tamaño de las variables se calcula en tiempo de compilación. Las colecciones de datos excesivamente grandes pueden provocar errores por agotamiento del espacio disponible. En la figura se dispone de un ejemplo susceptible de agotar el espacio de la pila.

2.6. CWEs del montículo

```

int pow(int base, int exp) {
    if (exp > 0) {
        return base * pow(base,
                           exp - 1);
    } else {
        return 1;
    }
}

```

```

pow(5, 4)
5 * pow(5, 3)
5 * (5 * pow(5, 2))
5 * (5 * (5 * pow(5, 1)))
5 * (5 * (5 * (5 * pow(5, 0))))
5 * (5 * (5 * (5 * 1)))
625

```

Figura 2: A la izquierda se tiene el código fuente, a la derecha, el contenido de la pila en el nivel más profundo de recursión. En este caso, el calculo se expande según se profundiza. En situaciones de recursividad muy profunda, aumenta el riesgo de desbordamiento de pila por agotamiento de espacio. Ejemplo extraído de Wikipedia [7].

3

Metodología de análisis

4

Entrenamiento del gran modelo de lenguaje

5

Resultados recopilados

6

Conclusiones

La conclusión

Referencias

- [1] Robert C. Seacord. *Secure coding in C and C++*. eng. 2nd ed. SEI series in software engineering Secure coding in C and C++. Place of publication not identified: Addison Wesley, 2013. ISBN: 9780132981972.
- [2] Vahid Eftekhari Moghadam et al. “Memory Integrity Techniques for Memory-Unsafe Languages: A Survey”. En: *IEEE Access* 12 (2024), págs. 43201-43221. doi: 10.1109/ACCESS.2024.3380478.
- [3] Cristina Cifuentes et al. “The role of program analysis in security vulnerability detection: Then and now”. En: *Computers & Security* 135 (2023), pág. 103463. ISSN: 0167-4048. doi: <https://doi.org/10.1016/j.cose.2023.103463>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404823003735>.
- [4] Bushra Aloraini et al. “An empirical study of security warnings from static application security testing tools”. En: *Journal of Systems and Software* 158 (2019), pág. 110427. ISSN: 0164-1212. doi: <https://doi.org/10.1016/j.jss.2019.110427>. URL: <https://www.sciencedirect.com/science/article/pii/S0164121219302018>.
- [5] Ruohao Guo Jenny Chen. *CS 225 | Stack and Heap Memory*. URL: <https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/>.
- [6] Avd. *C/C++ maximum stack size of program on mainstream OSes*. 2009. URL: <https://stackoverflow.com/questions/1825964/c-c-maximum-stack-size-of-program-on-mainstream-oses>.
- [7] Wikipedia contributors. *Stack overflow — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Stack_overflow&oldid=1351011401. [Online; accessed 24-May-2026]. 2026.
- [8] Kamran Amini. *Extreme C : taking you to the limit in concurrency, OOP, and the most advanced capabilities of C*. eng. 1st edition. Birmingham: Packt, 2019. ISBN: 9781789341355.
- [9] Cal Erickson. *Memory leak detection in embedded Systems | Linux Journal*. 2002. URL: <https://www.linuxjournal.com/article/6059>.